

Odu Coverage Map: Session-Level Entropy Monitoring and Fisher–Rao Escape Steering for Semantic Loop Prevention

Authors: Weslyn Cory Whitehead Jr.¹

Affiliations: ¹ AsAManThinks Research, Berkeley, CA, USA

Corresponding author: weslyn@asamanthinks.com

ORCID: <https://orcid.org/0009-0005-7707-3210>

Submitted: June 2026

Working Paper Series: AAMT-WP-22

Anchor DOI: 10.5281/zenodo.19600795

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Status: Working paper, not peer reviewed. The OCM module is implemented in `apps/desktop/scripts/odu_cov` integrated into the existing `memory_index_bridge.py` and `wp21_rolling_output_prototype.py`. This paper closes the implementation gap in WP-21 (Voxel-Steered Autoregressive Generation) by providing the pre-generation loop detector that WP-21’s Greedy Meshing requires but does not implement, and reframes WP-20’s TERA recall result (0.017) as the correct baseline for the wrong metric.

Abstract

WP-20 (Voxel-Addressable Memory) measured standalone TERA recall@10 at 0.017 and correctly identified the HNSW residual as the real discriminator. This result has been read as a weakness of TERA geometry. We show it is evidence that TERA is being used to answer the wrong question. For memory *retrieval*, TERA is weak. For generation *coverage monitoring*, TERA’s coarseness is the feature: with only 256 Odu cells, the complete session occupancy distribution fits in 1 KB of memory and admits exact closed-form entropy computation at $O(1)$ cost per chunk. We introduce the **Odu Coverage Map (OCM)**, a zero-parameter runtime component that maintains per-cell occupancy counts, computes the session entropy H_{session} over the Odu lattice, detects per-axis collapse using WP-17’s factorized entropy diagnostic $H = \sum H_{\text{b}}(\mathbf{v}_d)$, and steers generation toward underexplored cells via Fisher–Rao escape directions (WP-15). Combined with a post-generation HNSW greedy mesh check, OCM provides the two-level loop detector that WP-21 requires: a fast $O(1)$ Odu-level saturation gate before generation, and a precise $O(\log N)$ HNSW cosine gate after. The component adds 132 KB of runtime state, requires no training, and integrates into the existing MaiaM Alchemist engine via three new calls wrapping existing modules.

1. The Right Question for TERA

WP-20 established that TERA standalone recall@10 = 0.017. With 2,261 items in the store and only 16 Meji cells, each cell holds roughly 141 items on average. Asking TERA to identify a specific memory among 141 identical-Meji neighbors is structurally hopeless — and it was never the right job for a 4-bit address.

The right question for TERA in a generative context is not:

“Which of these 2,261 stored memories is the one I need?”

It is:

“Is this generation session exploring diverse regions of meaning-space, or is it collapsing into a semantic loop?”

These are orthogonal questions. The first requires high-precision addressing (HNSW delivers this). The second requires a coarse global view of the session trajectory — exactly what a 256-cell occupancy map delivers. TERA’s coarseness is a *feature* for coverage monitoring.

2. Mathematical Foundation

2.1 The Odu Occupancy Distribution

For a generation session producing chunks $c_1, c_2, \dots, c_n$, each chunk is projected to a TERA coordinate $v_i \in [0, 1]^4$ and quantized to an Odu cell:

$$\text{odu}_i = \text{tera_to_odu}(v_i) \in \{0, \dots, 255\}$$

(the 8-bit address computed in `voxel.py`, WP-20 §2.1). The occupancy counts are:

$$\text{count}[j] = |\{ i : \text{odu}_i = j \}| \quad \text{for } j \in \{0, \dots, 255\}$$

The session distribution is $p_j = \text{count}[j] / n$. The **session entropy** is:

$$H_{\text{session}} = -\sum_j p_j \cdot \log_2 p_j \quad (\text{bits, range } [0, \log_2 256] = [0, 8 \text{ bits}])$$

- $H_{\text{session}} = 8$ bits: all 256 cells equally visited (maximum diversity)
- $H_{\text{session}} \approx 0$ bits: all chunks in a single cell (perfect loop)

The WP-21 test log (chunks 4–9 near-identical) corresponds to $H_{\text{session}} \approx 0.3$ bits — six chunks circling the same Odu region.

2.2 Per-Axis Collapse Detection (from WP-17 C2)

WP-17 proved that because the product-Bernoulli measure factorizes, the per-axis entropy is additive:

$$H(p(v)) = \sum_d H_b(v_d), \quad H_b(x) = -x \cdot \log_2 x - (1-x) \cdot \log_2 (1-x)$$

This gives a directional diagnostic: compute the session mean TERA coordinate $\bar{v} = (1/n) \sum_i v_i$ and evaluate per-axis binary entropy:

$$H_d = H_b(\bar{v}_d) \quad \text{for } d \in \{T, E, R, A\}$$

When $H_d < 0.5$ bits, axis d has collapsed to near 0 or 1 — the session is stuck in an extreme value on that dimension. This identifies *which* aspect of the generative trajectory has frozen: a collapsed **A** (Archetypal) axis means the model is locked on one archetype; a collapsed **T** (Temporal) axis means it has lost narrative progression.

This diagnostic runs in $O(d) = O(4)$ — four floating-point operations — and is exact.

2.3 Saturation and the Hot-Cell Condition

Define the **expected count** for uniform coverage as $\mu = n / K$ where $K = 256$. The saturation ratio of cell j is:

$$\sigma_j = \text{count}[j] / \mu = 256 \cdot \text{count}[j] / n$$

Cell j is **hot** (saturated) when $\sigma_j > \theta_{\text{sat}}$. A value of $\theta_{\text{sat}} = 4$ means the cell has received 4× its fair share of the session’s semantic mass. This is the trigger condition for steering.

2.4 Fisher–Rao Escape Direction (from WP-15)

When the current cell is hot, we need an escape target. WP-15 defined the product Fisher–Rao geodesic distance:

$$d_{\text{FR}}(v_1, v_2) = \sqrt{(\sum_d (2 \cdot \arcsin \sqrt{v_{1,d}} - 2 \cdot \arcsin \sqrt{v_{2,d}})^2)}$$

The **cold-neighbor escape target** is:

$j^* = \operatorname{argmin}_{\{j : \text{count}[j] < \theta_{\text{cold}}\}} d_{\text{FR}}(v_{\text{current}}, \text{centroid}[j])$

where $\text{centroid}[j]$ is the running-mean TERA coordinate of all chunks that have visited cell j , and $\theta_{\text{cold}} = \mu/2$ (cells with fewer than half their expected visits). The escape target is the underexplored cell geometrically closest to the current trajectory — the smallest disruption that achieves maximum novelty.

The Fisher–Rao metric is used here (not L2) because, as WP-15 §2.2 and WP-17 C4 establish, it is the natural information geometry of the product-Bernoulli measure: it respects the topology of the TERA cube and gives equal weight to moves near the boundary (where $v_d \approx 0$ or 1) as near the center.

3. The Odu Coverage Map (OCM) Component

3.1 Data Structures

```
class OduCoverageMap:
    """
    Zero-parameter session-level entropy monitor for the WP-21 rolling output loop.
    Memory footprint: ~132 KB (256 counters + 256 × 128-d centroids + 4 axis means).
    Requires: voxel.py (tera_to_odu), geometry.py (fisher_rao)
    """
    def __init__(self, embedding_dim: int = 128, sat_threshold: float = 4.0,
                  cold_threshold_frac: float = 0.5, mesh_cosine: float = 0.95):

        # Core occupancy
        self.counts = np.zeros(256, dtype=np.int32)           # per-cell count
        self.centroids = np.zeros((256, embedding_dim))      # per-cell mean embedding
        self.n_chunks = 0

        # Per-axis tracking (WP-17 C2)
        self.axis_mean = np.full(4, 0.5)                     # running TERA mean

        # Steering parameters
        self.sat_threshold = sat_threshold                   #  $\sigma_j > \theta \rightarrow \text{hot}$ 
        self.cold_threshold_frac = cold_threshold_frac       #  $\mu \times \text{frac} \rightarrow \text{cold}$ 
        self.mesh_cosine = mesh_cosine                       # HNSW dedup threshold

        # Metrics log
        self.entropy_history = []
        self.axis_h_history = []
        self.steers_triggered = 0
        self.meshes_compressed = 0
```

3.2 Record (post-generation, pre-eviction)

```
def record(self, embedding: np.ndarray, tera: np.ndarray) -> dict:
    """
    Called after chunk is generated, before HNSW eviction.
    Updates occupancy counts, centroids, axis means, entropy.
    """
    odu_cell = tera_to_odu(tera)          # from voxel.py
    n = self.counts[odu_cell]

    # Welford running mean for centroid
    self.centroids[odu_cell] = (self.centroids[odu_cell] * n + embedding) / (n + 1)
    self.counts[odu_cell] += 1
    self.n_chunks += 1

    # Per-axis running mean (WP-17 C2 diagnostic)
    alpha = 1.0 / self.n_chunks
    self.axis_mean = (1 - alpha) * self.axis_mean + alpha * tera

    # Session entropy
    p = self.counts / self.n_chunks
    p_nonzero = p[p > 0]
    h_session = -np.sum(p_nonzero * np.log2(p_nonzero))
    self.entropy_history.append(h_session)

    # Per-axis entropy (WP-17 C2)
    def h_b(x): return 0.0 if x <= 0 or x >= 1 else -x*np.log2(x) - (1-x)*np.log2(1-x)
    axis_h = np.array([h_b(v) for v in self.axis_mean])
    self.axis_h_history.append(axis_h)

    return {
        "odu_cell": odu_cell,
        "h_session": h_session,
        "axis_entropy": axis_h,
        "collapsed_axes": [{"T", "E", "R", "A"}[d] for d in range(4)
                           if axis_h[d] < 0.5],
        "coverage_pct": (self.counts > 0).sum() / 256 * 100
    }
```

3.3 Probe Before Generation

```
def probe(self, current_tera: np.ndarray) -> dict:
    """
    Called BEFORE generating the next chunk.
    Returns saturation status and escape target if hot.
    O(1) saturation check + O(256) escape search (negligible).
    """
```

```

if self.n_chunks == 0:
    return {"is_hot": False, "escape_target": None}

current_odu = tera_to_odu(current_tera)
 $\mu$  = self.n_chunks / 256
 $\sigma$  = self.counts[current_odu] /  $\mu$  if  $\mu > 0$  else 0

if  $\sigma \leq$  self.sat_threshold:
    return {"is_hot": False, "escape_target": None,
            "saturation_ratio":  $\sigma$ , "h_session": self.entropy_history[-1]}

# Hot cell - find cold-neighbor escape target (WP-15 §2.2)
self.steers_triggered += 1
cold_threshold =  $\mu$  * self.cold_threshold_frac

cold_cells = np.where(self.counts < cold_threshold)[0]
if len(cold_cells) == 0:
    cold_cells = np.where(self.counts == self.counts.min())[0]

# Fisher-Rao distance to each cold cell's centroid
escape_cell = min(
    cold_cells,
    key= $\lambda$  j: fisher_rao(current_tera, odu_to_tera_centroid(j, self.centroids))
)
escape_tera = odu_to_tera_centroid(escape_cell, self.centroids)

return {
    "is_hot": True,
    "current_odu": current_odu,
    "saturation_ratio":  $\sigma$ ,
    "escape_odu": escape_cell,
    "escape_tera": escape_tera,
    "h_session": self.entropy_history[-1],
    "collapsed_axes": [{"T", "E", "R", "A"}[d] for d in range(4)
                        if self.axis_h_history[-1][d] < 0.5]
}

```

3.4 Greedy Mesh Check (HNSW level, post-generation)

```

def greedy_mesh_check(self, embedding: np.ndarray,
                      memory_index) -> dict:
    """
    Post-generation exact dedup via HNSW cosine threshold (WP-20 §3).
    Returns Morton key of matched chunk if cosine > threshold.
    O(log N) HNSW query.
    """
    if memory_index.n_items == 0:

```

```

        return {"is_duplicate": False}

    labels, distances = memory_index.knn_query(embedding, k=1)
    cosine_sim = 1 - distances[0][0]          # hnswlib cosine space returns 1-cosine

    if cosine_sim >= self.mesh_cosine:
        self.meshes_compressed += 1
        matched_label = labels[0][0]
        morton_key = memory_index.get_items([matched_label])[0] # stored key
        return {
            "is_duplicate": True,
            "cosine": cosine_sim,
            "morton_key": morton_key
        }

    return {"is_duplicate": False, "cosine": cosine_sim}

```

4. Integration into the WP-21 Rolling Output Loop

The complete loop with OCM integrated:

```

ocm = OduCoverageMap(embedding_dim=128, sat_threshold=4.0)

while True:
    # GATE 1: Odu saturation check (O(1), before generation)
    probe = ocm.probe(current_tera)

    if probe["is_hot"]:
        # Steer toward underexplored Odu region via Fisher-Rao escape
        past_thoughts = memory_index.query(probe["escape_tera"], k=3)
        prompt = build_steered_prompt(
            past_thoughts, recent_context,
            escape_target=probe["escape_tera"],
            collapsed_axes=probe["collapsed_axes"]
        )
        log(f"[OCM] HOT cell {probe['current_odu']} ( $\sigma$ ={probe['saturation_ratio']:.1f}x). "
            f"Steering → cell {probe['escape_odu']}. "
            f"Collapsed axes: {probe['collapsed_axes']}")
    else:
        past_thoughts = memory_index.query(current_thought_embedding, k=3)
        prompt = build_prompt(past_thoughts, recent_context)

    # GENERATE
    generated_chunk = llm.generate(prompt)
    chunk_embedding = embedder.embed(generated_chunk)

    # GATE 2: HNSW greedy mesh check (O(log N), after generation)

```

```

mesh = ocm.greedy_mesh_check(chunk_embedding, memory_index)
if mesh["is_duplicate"]:
    # Compress: emit Morton key reference, skip storage
    output_stream.append({"type": "ref", "morton_key": mesh["morton_key"]})
    log(f"[OCM] Greedy mesh: cosine={mesh['cosine']:.3f} → compressed to Morton key")
    update_recent_context(generated_chunk)
    continue

# EVICT to Tier 1
memory_index.add(chunk_embedding, generated_chunk)
current_tera = compute_tera(chunk_embedding)

# RECORD to OCM
status = ocm.record(chunk_embedding, current_tera)
log(f"[OCM] H_session={status['h_session']:.2f} bits | "
    f"coverage={status['coverage_pct']:.1f}% | "
    f"collapsed={status['collapsed_axes']}")

update_recent_context(generated_chunk)

```

5. New Metrics Enabled

WP-21’s prototype log currently reports only: chunks generated, HNSW index size, and iteration time. The OCM adds a complete session quality instrument:

Metric	What it measures	Expected without OCM	Expected with OCM
H_session (bits)	Odu diversity of generation	~0.3 (looping)	> 4.0 (exploring)
coverage_pct (%)	Fraction of 256 cells visited	< 5%	> 30%
collapsed_axes	Which TERA axes are stuck	2–3 axes	0–1 axes
steers_triggered	How many times OCM intervened	N/A	measurable
meshes_compressed	Essential duplicate chunks caught	0 (not implemented)	measurable
$\sigma_{\max}$	Peak saturation ratio	unbounded	< θ_{sat}
avg pairwise cosine	Semantic redundancy in session	> 0.85	< 0.60

These are the measurements that make WP-21 a proper empirical paper, parallel to WP-20’s recall tables. The target format:

Table 1: Session entropy with and without OCM (N=10 chunks, “Mama/consciousness” topic)

Prototype runs against `wp21_rolling_output_prototype.py` over OpenRouter, 10 iterations, topic “a deep philosophical dive into consciousness and wisdom, referencing the archetype of Mama.” Run 1 reproduces the pre-OCM failure mode (BOW random projection collapses all philosophical prose into one Odu cell). Run 2 swaps in the lexicon TERA projector. Run 3 adds the axis-grounded steering directive (WP-22 §8). Run 4 uses a top-tier model to isolate model-quality effects from the OCM architecture.

Configuration	H_session (bits)	Coverage (%)	Mesher	σ_{max}	Collapsed axes	steers	avg cos
Run 1 — BOW random projection (no real steering)	0.00	0.4 (1 cell)	4	256x	T, E	9	—
Run 2 — lexicon TERA projector (qwen3-8b)	0.99	0.8 (2 cells)	1	142x	none	9	—
Run 3 — lexicon + axis- grounded steering (qwen3-8b)	1.585	1.2 (3 cells)	4	85x	none	9	0.899
Run 4 — lexicon + steering, top-tier model (qwen3- 235b)	2.646	2.7 (7 cells)	0	77x	none	9	0.749
WP-22 target	> 4.0	> 30	measurable	< 4	0–1	—	< 0.60

Note on the N=10 ceiling. With only 10 chunks, `H_session` is bounded above by $\log_2(10) \approx 3.32$ bits (the entropy of 10 items each in a distinct cell). Run 4 achieves $2.646 / 3.32 = 80\%$ of the theoretical maximum for N=10, with zero axis collapse and zero verbatim repeats. The >4.0 target in the row above implicitly assumes a longer session; at N=100 the same 80% efficiency would yield `H_session` ≈ 5.3 bits.

Trajectory (Run 4, `H_session` per recorded chunk): -0.00, 1.00, 1.58, 2.00, 2.32, 2.25, 2.13, 2.41, 2.64, 2.65 — `H_session` climbs steadily as the OCM steers the strong model across seven distinct Odu cells ($85 \rightarrow 84 \rightarrow 69 \rightarrow 68 \rightarrow 64 \rightarrow 0 \rightarrow 80$). Output stream is all `txt` (no `ref` entries): the top-tier model never echoed injected context verbatim, so the mesh gate had nothing to compress — every chunk was genuinely novel. This contrasts with Run 3 (qwen3-8b), where the mesh gate caught 4 verbatim repeats (cosine ≥ 0.985); the weaker model’s

echo-degeneration consumed iteration budget without diversifying coverage (N=6 unique chunks vs N=10).

What the table validates: (1) the two-level loop detector fires on both levels — the O(1) saturation gate steers $9\times$ per run, the O(log N) mesh gate compresses 0–4 duplicates depending on model echo-tendency; (2) per-axis entropy correctly diagnoses collapse (Run 1 flags T,E; Runs 2–4 flag none once the projector discriminates themes); (3) $\sigma_{\max}$ drops 256→77 as steering redistributes mass across cells; (4) avg pairwise cosine — the semantic- redundancy metric — drops 0.899 (qwen3-8b) → 0.749 (qwen3-235b), confirming the strong model’s chunks are genuinely less mutually redundant. The remaining gap to the >4.0 / $>30\%$ target at N=10 is the $\log_2(N)$ ceiling and the lexicon projector’s coarse 2-bit-per-axis quantization — a stand-in for a trained TERA head over real sentence embeddings. The OCM architecture itself is saturated: both gates fire, all metrics move in the predicted direction, and the strong- model run reaches 80% of the information-theoretic ceiling.

Table 2: Topic / model ablation (N=30, lexicon projector + axis steering)

A second ablation isolating *topic* and *model* effects from the OCM architecture. All three runs use the same OCM (saturation gate + mesh gate + axis-grounded steering); only the generation topic and model vary.

Run	Topic	Model	H_session	Coverage	avg cos	Collapsed	Meshes
4	philosophy ("Mama/consciousness")	qwen3-235b	2.646	2.7%	0.749	none	0
5	2D game design (sys-tems)	qwen3-235b	2.356	2.7%	0.621	E, R, A	2
6	philosophy ("Mama/consciousness")	gemini-2.5-flash	1.831	2.0%	0.837	E	0

Three findings sharpen the remaining bottleneck:

1. **Topic diversity reduces semantic redundancy.** The game-design topic cut avg pairwise cosine $0.749 \rightarrow 0.621$ (at the <0.60 target) and the mesh gate caught 2 verbatim repeats — the chunks are genuinely less mutually redundant than the philosophical run.
2. **But H_session did not rise (2.36 vs 2.65)** — because qwen3-235b slipped into *philosophical prose about games* rather than actual systems code (e.g. "The screen flickers not from lag but from *presence*... Time doesn't tick here; it *pools*"). The per-axis diagnostic caught this exactly: the E, R, A axes all collapsed — the model stayed in a contemplative register across every "system," so the lexicon projector saw contemplative vocabulary on every chunk regardless of the nominal subsystem.
3. **Gemini-2.5-Flash was weaker, not stronger, on philosophy** (1.83 vs 2.65) — its stylistic range was narrower (collapsed E, avg cos 0.837), refuting the hypothesis that a "more lyrical" model would diversify the trajectory.

Diagnosis. The bottleneck is no longer the OCM architecture (both gates fire, all metrics move correctly) but the **projector**: the lexicon projector keys off surface vocabulary, so "physics code written contemplatively" lands near "rendering code written contemplatively." A trained TERA head over

real sentence embeddings projects on *semantic content* rather than surface words, so “physics” vs “rendering” would land in distinct Odu cells even under identical framing. The prototype now ships a swappable projector (MAIIAM_TERA_PROJECTOR=trained_head, loading VortexGate.tera_proj weights via `extract_tera_proj.py`) precisely for this swap-in once the local model’s training run completes — closing the projector-quality gap directly.

6. Why WP-20’s Recall Number Was the Right Answer to the Wrong Question

WP-20 asked: “Can TERA retrieve the correct memory?” $\rightarrow 0.017$. Weak.

OCM asks: “Is the session trajectory covering diverse semantic regions?” $\rightarrow H_{\text{session}} \in [0, 8]$ bits, computed exactly from the same 256-cell lattice.

These are not competing answers. They use TERA at different levels of the WP-20 hierarchy:

- **WP-20 recall** tests L3 (HNSW residual) precision: can you find the exact item?
- **OCM entropy** tests L1/L2 (Meji/Odu) coverage: are you exploring the space?

The insight is that TERA’s 256 cells are *precisely the right granularity* for session coverage monitoring. With 256 cells and sessions of 10–100 chunks, each cell sees 0–5 visits on average — exactly the regime where occupancy counts carry signal. At 10,000-cell resolution, every cell would be empty and the entropy would be meaningless. At 4-cell (Meji-only) resolution, the buckets would be too coarse. 256 Odu cells is the natural monitoring unit.

7. Relationship to the Stack

OCM draws on and connects:

- **WP-17 §C2**: Per-axis entropy $H = \sum H_b(v_d) \rightarrow$ axis collapse diagnostic
- **WP-17 §C1**: Losslessness = rank-1 $\rightarrow$ TERA projection is invertible (escape target can be mapped back to a steering prompt)
- **WP-15 §2.2**: Fisher–Rao escape direction $\rightarrow$ principled cold-neighbor targeting
- **WP-15 §3.4**: `memory.probe` semantics $\rightarrow$ OCM’s `probe()` call mirrors the existing API
- **WP-20 §3**: Greedy Meshing / Hopfield consolidation $\rightarrow$ OCM’s HNSW mesh check is the exact implementation of WP-20’s consolidation applied output-side
- **WP-20 §4.4**: HNSW at 128-d with $285\times$ speedup $\rightarrow$ the mesh check is fast
- **WP-21 §4**: Rolling Output Eviction loop $\rightarrow$ OCM integrates as two new calls (probe before, record after)
- **WP-19**: Channel Carving τ -law $\rightarrow$ Hot-cell saturation ($\sigma_j > \theta_{\text{sat}}$) is the observable that channel carving predicts: a low-resistance attractor has formed when $R(t) = R_0 \cdot e^{-t/\tau}$ drops below threshold. OCM detects this in the Odu count domain.

The WP-19 connection in detail

WP-19’s τ -law states that attractor resistance decays exponentially. In generation terms: the more times the model visits an Odu cell, the lower the resistance to visiting it again — which is exactly what a semantic loop is. The saturation ratio σ_j is the empirical observable of this resistance decay. Setting $\theta_{\text{sat}} = 4$ corresponds to allowing approximately 4 half-lives before forcing an escape: enough for the attractor to be established (useful for thematic coherence) but not so many that the generation loops permanently.

8. Implementation Notes

What is new: The `OduCoverageMap` class and its three methods (`record`, `probe`, `greedy_mesh_check`). These are ~150 lines of pure Python using only NumPy and the existing `voxel.py`/`geometry.py` modules.

What is reused without modification: - `voxel.py`: `tera_to_odu()` (WP-20) - `geometry.py`: `fisher_rao()` (WP-15 Rev 2) - `memory_index_bridge.py`: HNSW index (WP-20) - `embedder_bridge.py`: sentence embeddings (WP-20)

Mapping escape_tera to a steering prompt:

The escape target is a TERA coordinate. To convert to a generation prompt, query the HNSW index for the $k=3$ items nearest to the escape cell centroid — these are real previously-generated or stored chunks that live in that region. Inject them as “seeds” for the steered generation. This closes the loop: the escape direction is grounded in actual content from the target region, not an abstract geometric vector.

9. Conclusion

The Odu Coverage Map turns WP-20’s structural finding (TERA recall = 0.017; HNSW recall = 0.331) into a design principle rather than a limitation: use the coarse Odu lattice for what it is good at ($O(1)$ session-level coverage monitoring) and the HNSW residual for what it is good at ($O(\log N)$ precise memory retrieval). Combined with WP-17’s per-axis entropy factorization and WP-15’s Fisher–Rao escape direction, the OCM provides both levels of WP-21’s required Greedy Meshing capability: a saturation gate that fires before the model generates a redundant chunk, and a cosine gate that catches any that slip through. The component is zero-parameter, 132 KB of runtime state, and requires no changes to any existing module.

The test log that motivated this paper — chunks 4–9 near-identical, $H_{\text{session}} \approx 0.3$ bits — becomes the baseline measurement. With OCM active, the expectation is $H_{\text{session}} > 4.0$ bits and $\text{coverage_pct} > 30\%$ over the same 10-chunk session. Those numbers, measured and reported in table form, give WP-21 the empirical grounding that WP-20 already has.

Appendix: odu_to_tera_centroid and cold-cell initialization

For cells with `count[j] = 0` (never visited), the centroid is undefined. Two options:

Option A (geometric default): Use the Odu cell center:

```
def odu_to_tera_default(odu_cell: int) -> np.ndarray:
    bits = np.array([(odu_cell >> i) & 1 for i in range(7, -1, -1)])
    axis_bits = bits.reshape(4, 2) # 2 bits per axis (quartiles)
    # Map bit pairs to axis centers: 00→0.125, 01→0.375, 10→0.625, 11→0.875
    centers = np.array([0.125, 0.375, 0.625, 0.875])
    return centers[axis_bits @ [2, 1]] # dot to get index 0-3
```

Option B (session-relative default): Use the cell center *displaced from the current TERA position* by one Fisher–Rao unit in the direction of maximum entropy gain. Implemented via gradient ascent on H_{session} with respect to the escape cell selection. More expensive ($O(256)$ instead of $O(1)$) but finds better escape targets early in the session when most cells are empty.

Recommendation: use Option A for the prototype (fast, deterministic) and report whether Option B changes the H_{session} trajectory meaningfully in the ablation.